

Bipedal Robot Walking by Reinforcement Learning in Partially Observed Environment

U. Özalp¹, Ö. Uğur²

Abstract. Deep Reinforcement Learning methods on mechanical control have been successfully applied in many environments and used instead of traditional optimal and adaptive control methods for some complex problems. However, Deep Reinforcement Learning algorithms do still have some challenges. One is to control on partially observable environments. When an agent is not informed well of the environment, it must recover information from the past observations. In this thesis, walking of Bipedal Walker Hardcore (OpenAI GYM) environment, which is partially observable, is studied by two continuous actor-critic reinforcement learning algorithms; Twin Delayed Deep Deterministic Policy Gradient and Soft Actor Critic. Several neural architectures are implemented. The first one is Residual Feed Forward Neural Network under the observable environment assumption, while the second and the third ones are Long Short Term Memory and Transformer using observation history as input to recover the hidden information due to the partially observable environment.

Keywords. deep reinforcement learning, partial observability, robot control, actor-critic methods, long short term memory, transformer

★ Submitted to IAM, *Scientific Computing* in July 2021.

DRAFT OF A MANUSCRIPT

Student : Uğurcan Özalp

Advisor : Prof. Dr. Ömür Uğur

¹Middle East Technical University, Institute of Applied Mathematics, 06800 Çankaya, Ankara, Turkey.
E-Mail: ugurcan.ozalp@metu.edu.tr

²Middle East Technical University, Institute of Applied Mathematics, 06800 Çankaya, Ankara, Turkey.
E-Mail: ougur@metu.edu.tr

Bipedal Robot Walking by Reinforcement Learning in Partially Observed Environment

U. Özalp³, Ö. Uğur⁴

Contents

1 Introduction

Humans and animals exhibit several different behaviours in terms of interaction with environment, such as utterance and movement. Their behavior is based on past experience, the situation they are in and their objective. Like humans and animals, an intelligent agent is expected to take action according to its perception based on some objective. A major challenge in Machine Learning to create agents that will act more natural and humanlike. As a subfield of ML, Reinforcement Learning (RL) allows an agent to learn how to control (or act) itself in different situations by interacting with the environment. In RL, environment is modeled to give reward (or punishment) to agent according to environmental state and agent actions, and agent focuses on learning to predict what actions will lead to highest reward (or lowest punishment, based on its objective) in the future using past experience.

Traditional RL algorithms need feature engineering from observations. For complex problems, the way to extract features is ambiguous or observations are not enough to create a good model. In contrast to traditional methods, Deep Neural Networks (DNNs) allows to extract high level features from data with large state-space (pixelwise visual, many kinematic sensors etc.) and missing observations. Along with recent developments in DNNs, Deep Reinforcement Learning (DRL) allows an agent to interact with the environment in a more complex way.

Since its discovery, robots have been crucial devices for the human race, whether smart or not. Intelligent humanoid and animaloid robots have been developed since early 1980s. This type of robots has legs unlike driving robots. Since most of the world terrain is unpaved, this type of robots are good alternative to driving robots. Locomotion is a major task for such robots. Stable bipedal (2 legged) walking is one of the most challenging problem among the control problems. It is hard to create accurate model due to high order of dynamics, friction and discontinuities. Even further, the design of walking controller using traditional methods is difficult due to the same reasons. Therefore, for bipedal walking, DRL approach is an easier choice if a simulation environment is available.

In this work, Bipedal Locomotion is investigated through *BipedalWalker-Hardcore-v3* [?] environment of open source GYM library [?] by DRL. Our contributions are as follows:

³Middle East Technical University, Institute of Applied Mathematics, 06800 Çankaya, Ankara, Turkey.
E-Mail: ugurcan.ozalp@metu.edu.tr

⁴Middle East Technical University, Institute of Applied Mathematics, 06800 Çankaya, Ankara, Turkey.
E-Mail: ougur@metu.edu.tr

- Sequential neural networks are used (LSTM and Transformer) for solution, along with Residual Feed Forward Neural Network.
- Twin Delayed Deep Deterministic Policy Gradient (TD3) and Soft Actor-Critic (SAC) algorithms are used and compared.
- Reward shaping and small modifications are applied on environment to hinder possible problems.

2 Environment Details

2.1 Dynamics

BipedalWalker environments [?, ?] are parts of Gym environment library. One of them is classical where the terrain is relatively smooth, while other is a hardcore version containing ladders, stumps and pitfalls in terrain. Robot has deterministic dynamics but terrain is randomly generated at the beginning of each episode. Dynamics of the robot are exactly identical in both environments. The robot has kinematic and lidar sensors.

Those environments have continuous action and observation space. **Observation Space** contains hull angle, hull angular velocity, hull translational velocities, joint positions, joint angular speeds, leg ground contacts and ten lidar rangefinder measurements. The robot has 2 legs with 2 joints at knee and hip. Torque provided to knee and pelvis joints of both legs. These 4 torque values forms **Action Space**.

Reward Function is designed to move the robot forward as much as possible. The robot should run fast with little energy while not stumble and fall to ground. Directly proportional to distance traveled forward, +300 points given if agent reaches end of path. -10 points (-100 points in original version) if agent falls, and small amount of negative reward proportional to absolute value of applied motor torque τ (to prevent applying unnecessary torque). Lastly, the robots gets negative reward proportional to absolute value of hull angle θ_{hull} for reinforcing to keep hull straight. Mathematically, reward is;

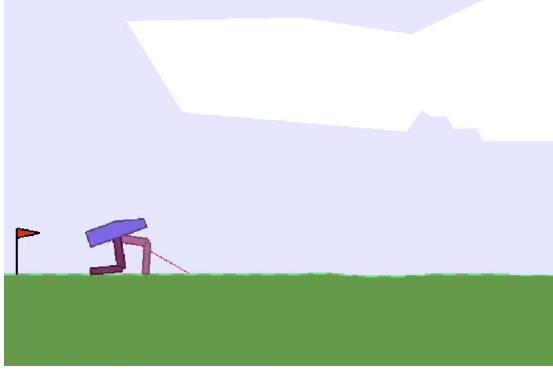
$$R = \begin{cases} -10, & \text{if falls,} \\ 130\Delta x - 5|\theta_{hull}| - 0.00035|\tau|, & \text{otherwise,} \end{cases} \quad (1)$$

Snapshots for both environments are depicted in Figure ??.

2.2 Difficulties

Locomotion of the Bipedal Walker is a difficult control problem due to following reasons:

- **Nonlinearity:** The dynamics are nonlinear, unstable and multimodal. Dynamical behavior of robot changes for different situations like ground contact, single leg contact and double leg contact.



(a) BipedalWalker-v3 Snapshot [?]



(b) BipedalWalkerHardcore-v3 Snapshot [?]

Figure 1: Bipedal Walkers Snapshots

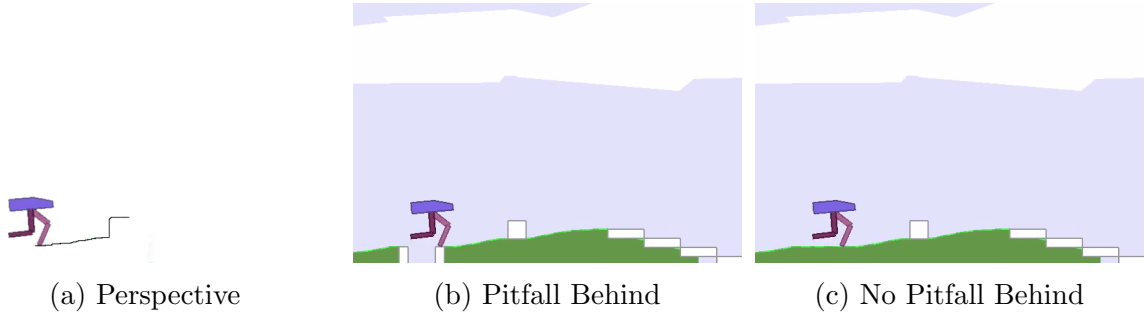


Figure 2: Perspective of agent and possible realities

- **Uncertainty:** The terrain where the robot walks also varies. Designing a controller for all types of terrain is difficult.
- **Reward Sparsity:** Overcoming some obstacles requires a specific maneuver, which is hard to explore sometimes.
- **Partially Observability:** The robot observes ahead with lidar measurements and cannot observe behind, as illustrated in Figure ???. In addition, it lacks of acceleration sensors.

These reasons make it also hard to implement analytical methods for control tasks. However, DRL approach can easily overcome nonlinearity and uncertainty problems.

On the other hand, reward sparsity problem brings local minimums to objective function of optimal control. However, this can be challenged by a good exploration strategy and reward shaping.

For the partial observability problem, more elegant solution is required. This is, in general, achieved by creating a belief state from the past observations to inform the agent. Agent uses this belief state to choose how to act. If the belief state is evaluated sufficiently, this increases performance of the agent. Relying on instant observations is also possible, and this may be enough sometimes if advanced type of control is not required.

3 Reinforcement Learning

Reinforcement Learning is the closest kind of learning demonstrated by humans and animals since it is grounded by biological learning systems. It is based on maximizing cumulative reward over time to make agent learn how to act in an environment [?]. Each action of the agent is either rewarded or punished according to a reward function. The agent explores environment by taking various actions in different states to gain experience, based on trial-and-error. Then it exploits experiences to get highest reward from the environment considering instant and future rewards over time.

RL is a stochastic control process. At time t , the agent starts with state $s_t \in \mathcal{S}$ and observes $o_t \in \mathcal{O}$, then it takes an action $a_t \in \mathcal{A}$ according to its policy π and obtains a reward $r_t \in \mathbb{R}$ at time t . Hence, a state transition to $s_{t+1} \in \mathcal{S}$ occurs as a consequence of the action and the agent gets the next observation $o_{t+1} \in \mathcal{O}$. Along with state transitions, the agent is expected to develop (learn) a policy function $\pi: \mathcal{S} \rightarrow \mathcal{A}$ which maps inputs (observations) $s \in \mathcal{S}$ to outputs (actions) $a \in \mathcal{A}$ to maximize rewards over time.

3.1 Policy Learning

Policy learning is achieved by maximization of the action value function, which is expected cumulative sum of future rewards scaled by discount factor γ . Action value function is defined as follows;

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{i=t}^{\infty} \gamma^{i-t} r_i | s_t = s, a_t = a, \pi \right]. \quad (2)$$

for all possible states and actions. Bellman proved that optimal value function, for a model T , should satisfy following conditions [?].

$$Q^*(s, a) = \mathbb{E}[r|s, a] + \gamma \max_{a'} \left\{ \sum_{s'} T(s'|s, a) Q^*(s', a') \right\} \quad (3)$$

Most of RL methods are build upon solving (??), since there exist a direct relation between Q^* and π^* , which is

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \arg \max_a Q^*(s, a), \\ 0, & \text{otherwise .} \end{cases} \quad (4)$$

3.2 Q Learning

Q Learning is based on optimizing Q function using Bellman Equation (??) [?]. In this learning strategy, Q function is assumed to be parametrized by θ . Target Q value is estimated by bootstrapping estimate,

$$Y_t^Q = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta). \quad (5)$$

At time t , with state, action, reward, next state tuples (s_t, a_t, r_t, s_{t+1}) , Q values are updated by minimizing difference between the target value and the estimated value. Hence the loss

$$\mathcal{L}_t(\theta) = (Y_t - Q(s_t, a_t; \theta))^2, \quad (6)$$

is to be minimized with respect to θ using numerical optimization methods.

3.3 Deep Q Learning

When a nonlinear approximator is used for Q estimation, learning is unstable due to the correlation among recent observations. Deep Q Learning solves this problem by introducing Target Network and Experience Replay [?, ?] along with using deep neural networks.

Target Network is parametrized by θ^- . It is used to evaluate target value, but it is not updated by the loss minimization. It is updated at each fixed number of update step by Q network parameter θ . In this way, correlation between target value and observations are reduced. The target value is obtained by using θ^- as

$$Y_t^{DQN} = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-). \quad (7)$$

Experience Replay stores experience tuples in the replay memory $\mathcal{D}$ as a queue with fixed buffer size N_{replay} . At each iteration i , θ is updated by experiences $(s, a, r, s') \sim U(\mathcal{D})$ uniformly subsampled from experience replay by minimizing the expected loss

$$\mathcal{L}_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(\mathcal{D})} \left[(Y^{DQN} - Q(s, a; \theta_i))^2 \right]. \quad (8)$$

It allows agent to learn from experiences multiple times at different stages of learning. More importantly, sampled experiences are close to be independent and identically distributed if buffer size is large enough. Again, this reduces correlation between recent observations and updated Q value. This makes learning process more stable.

Epsilon Greedy Exploration is used to let agent explore the environment. In discrete and finite action space $\mathcal{A}$, this is the simplest exploration strategy used in RL algorithms. During the learning process, a random action with probability ϵ , or greedy action (maximizing Q value) with probability $1 - \epsilon$ is selected. In order to construct policy:

$$\pi(a|s) = \begin{cases} 1 - \epsilon, & \text{if } a = \arg \max_a Q(s, a), \\ \frac{\epsilon}{|\mathcal{A}| - 1}, & \text{otherwise,} \end{cases} \quad (9)$$

where $|\mathcal{A}|$ denotes cardinality of $\mathcal{A}$.

3.4 Deep Deterministic Policy Gradient

DDPG is continuous complement of DQN using a deterministic policy [?]. It also uses experience replay and target networks. Similar to Deep Q Learning, there are target

networks parametrized by θ^{μ^-} and θ^{Q^-} along with the main networks parametrized by θ^μ and θ^Q .

While target networks are updated in a fixed number of steps in DQN, DDPG updates target network parameters at each step with Polyak averaging,

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-. \quad (10)$$

The τ is an hyperparameter indicating how fast the target network is updated and it is usually close to zero.

Policy network parameters are learned by maximizing resulting expected value, or minimizing its negative,

$$\mathcal{L}_i(\theta_i^\mu) = -\mathbb{E}_{s \sim U(\mathcal{D})} \left[Q(s, \mu(s; \theta_i^\mu); \theta^{Q^-}) \right]. \quad (11)$$

Note that value network parameters are also assumed to be learned.

In addition, target networks are used to predict target value and the target is defined by

$$Y_t^{DDPG} = r_t + \gamma Q(s_{t+1}, \mu(s_{t+1}; \theta^{\mu^-}); \theta^{Q^-}). \quad (12)$$

In each iteration, this target is used to learn Q function by minimizing the least squares loss

$$\mathcal{L}_i(\theta_i^Q) = \mathbb{E}_{s,a,r,s' \sim U(\mathcal{D})} \left[(Y^{DDPG} - Q(s, a; \theta_i^Q))^2 \right]. \quad (13)$$

In DDPG, value and policy network parameters are learned simultaneously. During the learning process, exploration noise is added to each selected action, sampled by a process $\mathcal{X}$. In [?], authors proposed to use Ornstein-Uhlenbeck Noise [?] in order to have temporal correlation for efficiency. However, a simple Gaussian white noise or any other one is also possible.

3.5 Twin Delayed Deep Deterministic Policy Gradient

TD3 [?] is an improved version of DDPG with higher stability and efficiency. There are three main tricks upon DDPG:

Target Policy Smoothing is to regularize the learning process by smoothing effects of actions on value. For target value assessing, actions are obtained from the target policy network in DDPG, while a clipped zero-centered Gaussian noise is added to actions in TD3 as follows:

$$\tilde{a}' = \mu(s'; \theta^{\mu^-}) + \text{clip}(\epsilon, -c, c), \quad \epsilon \sim \mathcal{N}(0, \sigma^2). \quad (14)$$

Clipped Double Q Learning is to escape from Q value overestimation. There are two different value networks with their targets. During learning, both networks are trained by single target value assessed by using whichever of the two networks give smaller. In other words,

$$Y_t^{TD3} = r_t + \gamma \min_{k \in \{1,2\}} Q(s_{t+1}, \tilde{a}_{j+1}; \theta^{Q_k^-}). \quad (15)$$

On the other hand, the policy is learned by maximizing the output of the first value network, or minimizing its negative,

$$\mathcal{L}_i(\theta_i^\mu) = -\mathbb{E}_{s \sim U(\mathcal{D})} \left[Q(s, \mu(s; \theta_i^\mu); \theta^{Q_1}) \right]. \quad (16)$$

Delayed Policy Updates is used for stable training. During learning, policy network and target networks are updated less frequently (at each fixed number of steps) than the value network. Since the policy network parameters are learned by maximizing the value network, they are learned slower.

3.6 Soft Actor-Critic

SAC [?] is a stochastic actor-critic method and many characteristics are similar to TD3 and DDPG. It is an entropy-regularized RL method. Such methods give bonus reward to the agent propotional to the policy entropy: given state s , the entropy of a policy π is defined by

$$H(\pi(\cdot|s)) = \mathbb{E}_{\tilde{a} \sim \pi(\cdot|s)} [-\log(\pi(\tilde{a}|s))] \quad (17)$$

Given the entropy coefficient α , definition of state-action value function is redefined as follows:

$$Q^\pi(s, a) = \mathbb{E}_{\substack{s' \sim T(\cdot|s, a) \\ \tilde{a}' \sim \pi(\cdot|s')}} \left[r + \gamma \left(Q^\pi(s', \tilde{a}') - \alpha \log(\pi(\tilde{a}'|s')) \right) \right]. \quad (18)$$

This modification changes Q value target definition,

$$Y_t^{SAC} = r_t + \gamma \left(\min_{k \in \{1, 2\}} Q(s_{t+1}, \tilde{a}_{t+1}; \theta^{Q_k^-}) - \alpha \log(\pi(\tilde{a}_{t+1}|s_{t+1})) \right), \quad (19)$$

where $\tilde{a}_{t+1} \sim \pi(\cdot|s_{t+1}; \theta^\pi)$.

While the policy is updated according to first value network by (??) in TD3, the policy is updated according to minimum value of both networks output along with the entropy regularization,

$$\mathcal{L}_i(\theta_i^\pi) = -\mathbb{E}_{s \sim U(\mathcal{D})} \left[\min_{\tilde{a} \sim \pi(\cdot|s)} \left[\min_{k \in \{1, 2\}} Q(s, \tilde{a}; \theta^{Q_k^-}) - \alpha \log(\pi(\tilde{a}|s; \theta_i^\pi)) \right] \right]. \quad (20)$$

The policy function is stochastic in SAC, and in practice, it is a parametrized probability distribution. Most common one is Squashed Gaussian policy to squash actions in the range $(-1, 1)$ by tanh function. This is parametrized by its mean and standart deviation, i.e., $\theta^\pi = (\theta^\mu, \theta^\sigma)$. Actions are then sampled as follows:

$$a(s) = \tanh(\mu(s; \theta^\mu) + \eta \odot \sigma(s; \theta^\sigma)), \quad \eta \sim \mathcal{N}(0, I), \quad (21)$$

where $\odot$ is elementwise multiplication.

Finally, we note that SAC method does not include policy delay, target policy smoothing and target policy network.

4 Methodology

4.1 Modifications on Original Environment

It is difficult to find an optimal policy directly with available setting. There are few works on the literature demonstrating a solution for *BipedalWalker-Hardcore-v3*. The following modifications are proposed.

- In original version, agent gets -100 points when its hull hits the floor. In order to make the robot more greedy, this is changed to -10 points. Otherwise, agent cannot explore environment since it gets too much punishment when failed and learns to do nothing.
- Time resolution of simulation is halved (from 50 Hz to 25 Hz) by only observing last of each two consecutive frames using a custom wrapper function. Since there is not a high frequency dynamics, this allows nothing but speeding up the learning process.
- In original implementation, an episode has a time limit. Once this limit is reached, simulation stops with terminal state flag. On the other hand, when agent fails before the time limit, the episode ends with terminal state flag too. In the first case, the terminal state flag causes instability since next step's value is not used in value update, since it is not a logical terminal, just time up indicator. The environment changed such that terminal state flag is not given in this case unless agent fails.

Once those modifications are applied on the environment, we observed quite improvement on both convergence and scores.

4.2 Proposed Neural Networks

Varying neural backbones used to encode state information from observations for both actor and critic networks. In critic network, actions are concatenated by state information coming from backbones. Then, this concatenated vector is passed through feed forward layer with hyperbolic tangent activation then through a linear layer with single output. Before feeding observations to backbone, they are passed through a linear layer with 96 dimensional output. However, for only LSTM backbones, this layer is not used and observations are passed to LSTM backbone as it is. In actor network, backbone is followed by a single layer with tanh activation for action estimation. Again, observations are passed through a linear layer with 96 dimensional output, and this is not valid for LSTM. Critic and Actor networks are illustrated in Figure ??

As backbones, following networks are proposed.

4.2.1 Residual Feed Forward Network

As Feed Forward Networks becomes deeper, optimizing weights gets difficult. Therefore, people come with the idea of residual connections [?]. For a fixed number of stacked layers

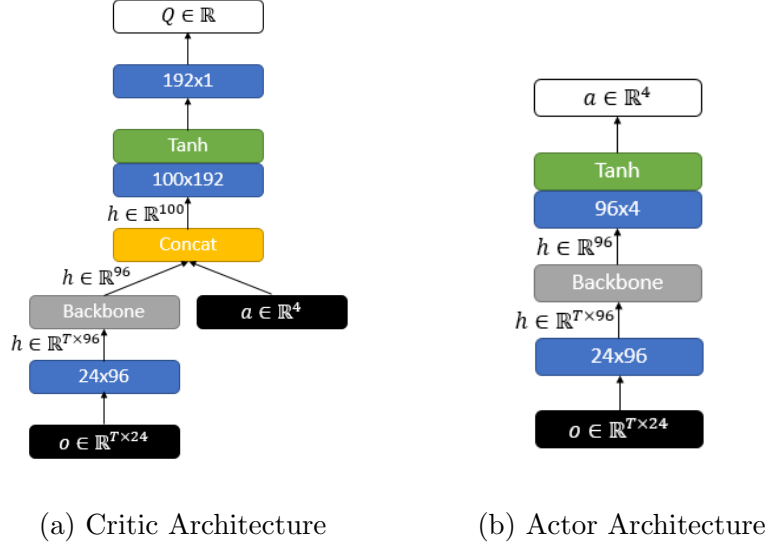


Figure 3: Neural Architecture Design

(usually 2 which is single skip), input and output of the stack is summed up for next calculations. Replacing feed forward layers with other types yield different kind of residual network. The difference is demonstrated in Figure ??, where ϕ corresponds to activation and yellow block means no activation.

In our application, incoming vector is passed through 2 layers with 192 dimensional hidden size and 96 dimensional output with single skip connection, where there is GELU activation at first layer.

4.2.2 Long Short Term Memory

Long Short Term Memory (LSTM) [?] is a special type of Recurrent Neural Network. It is explicitly designed to allow learning long-term dependencies [?]. A single LSTM cell has 4 neural layer while vanilla RNN layer has only one neural layer. In addition to hidden state h_t , there is another state called cell state C_t . Information flow is controlled by 3 gates.

In our application, sequence of incoming vectors is passed though single vanilla LSTM layer with 96 dimensional hidden state. Output at last time step is considered as belief state.

4.2.3 Transformer (Pre-layer Normalized)

The Transformer was proposed in Attention is All You Need [?] paper. Unlike recurrent networks, this architecture is solely built on attention layers. However, original transformer architecture includes layer normalization operations after attention and feed-forward layers. It is unstable since the values of gradients of output layers are high. Pre-Layer Normalized Transformer is proposed by [?] by carrying layer normalization operation

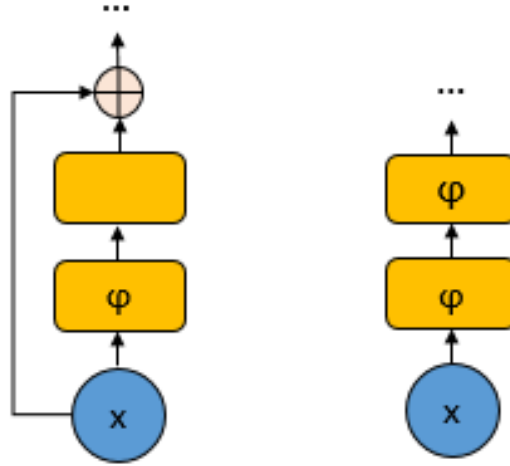


Figure 4: Deep Residual Feed Forward Network with single skip connection (left) and Deep Feed Forward Network (right)

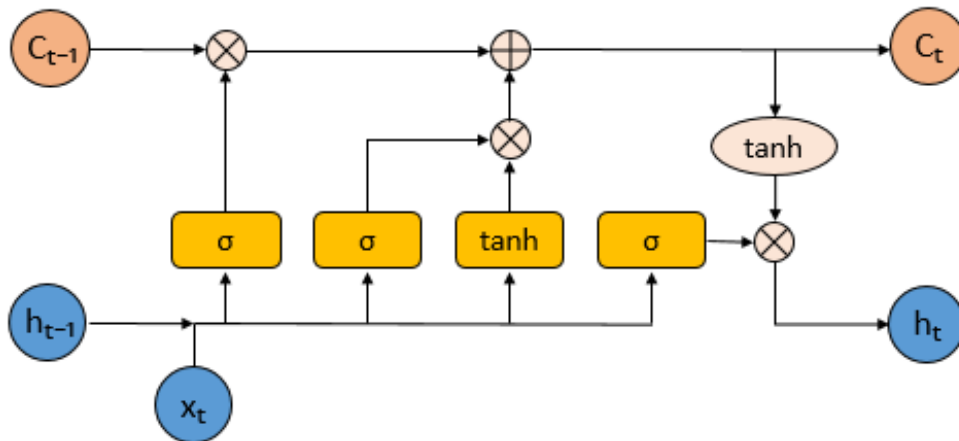


Figure 5: LSTM Cell

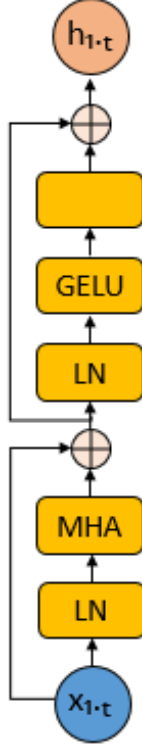


Figure 6: Pre-LN Transformer encoder layer with GELU activation

to in front of attention and feed-forward layers. Moreover, Parisotto et al. Xiong et al. [?] proposed gated transformer which also includes layer normalizations before attention and feedforward layer. They also stated that although gated architecture improves many RL tasks drastically, non-gated pre-layer normalized transformer are good enough. A simple pre-layer normalized transformer encoder layer is demonstrated in Figure ??.

In our application, sequence of incoming vectors is passed through single pre-layer normalized transformer layer with 192 dimensional feed forward layer with GELU activation after addition of positional encodings. In multi-head attention, 4 head is used and size of queris, keys and values are 8 for each head. During multi-head attention, only the last observation is fed as query so that attentions are calculated only for last state. Its output is considered as belief state. Note that excluding multi-head attention for this network gives our residual feed-forward neural network.

4.3 Summary of Networks

The designed networks have close number of parameters to make fair comparison among them. Number of trainable parameters for designed networks are summarized in Table ??.

In addition, we designed transformer network to be same as residual feed forward

Table 1: Number of trainable parameters of neural networks

Network	Actor Parameters	Critic Parameters
RFFNN	40512	59328
LSTM	47616	66432
TRSF	52992	71808

network once multi-head attention and preceding layer normalization is removed. It can be easily demonstrated by looking at Figure ?? (left one, layer normalization is added in the implementation) and Figure ??.

4.4 Algorithm Settings

TD3 and SAC is used as RL algorithm. All hyperparameters are found after a trial-error process considering the literature. Adam optimizer is used for optimization. For all models, agents are trained by 8000 episodes.

In TD3, as exploration noise, Ornstein-Uhlenbeck noise is used, and standart deviation is multiplied by 0.9995 at the end of each episode. For LSTM and Transformer, last 6 observations are used to train agent. More than 6 observations made TD3 algorithm diverge.

In SAC model, squashed gaussian policy is implemented. Therefore, along with layer giving mean actions, an additive layer is implemented for standart deviation along with it. For LSTM and Transformer, last 6 observations and 12 observations are used to train agent.

Training sessions are run by multiple times to make comparison fair among models since some training sessions fail to converge and some yield worse results.

4.5 Results and Discussion

For each episode, episode scores are calculated by summing up rewards of each time step. In Figure ?? and Figure ??, moving average and standard deviation is visualized for each model’s episode scores.

RFFNN seems enough to make agent walk, although there exist partial observability in the environment. That model reaches around 221 points in average with TD3 and 245 points with SAC.

Transformer models yield better performance compared to RFFNN. 249 points are reached by TD3 and 262 points reached by SAC model when 6 last observations are fed to the model. 266 points are obtained when last 12 observations are fed to model with SAC.

LSTM model yield the best results by reaching 264 points with TD3 and exceeds 280 points with SAC when 6 last observations are fed to model. 287 points are obtained when last 12 observations are fed to the model with SAC.

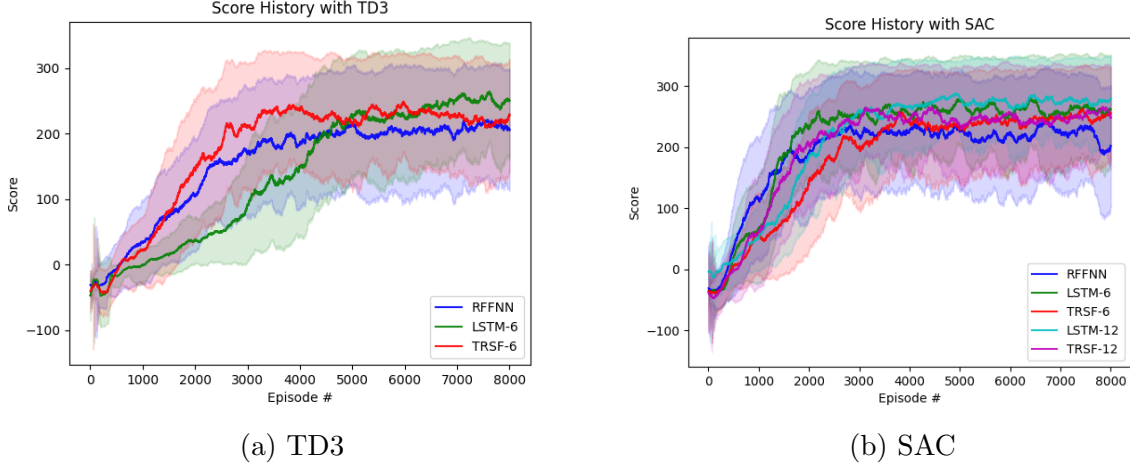


Figure 7: Moving Average and Standard Deviation for Episode Scores (Window length: 200 episodes) for TD3

Table 2: Best checkpoint performances with 100 test simulations

RL Method	Model	Episode	Avg. Score
TD3	RFFNN	6600	207
SAC	RFFNN	7600	219
TD3	TRSF-6	6400	222
SAC	TRSF-6	6800	254
SAC	TRSF-12	6000	270
TD3	LSTM-6	7000	242
SAC	LSTM-6	7600	268
SAC	LSTM-12	7200	298

These scores are maximum obtained average scores of concurrent 200 episodes during training. However, while training, model changes and agent still tries to explore environment. Officially, 300 points in average required in random 100 simulations to say the environment is solved [?]. Therefore, best checkpoints are evaluated for 100 concurrent simulations without exploration. Results are summarized in Table ??.

None of our models exceed 300 point limit, but LSTM-12 model with SAC almost reaches to the limit Figure ?. However, all models partially solved problems by exceeding 200 point limit, while some simulations exceed 300 points with both TD3 (Figure ?) and SAC (Figure ?).

Behaviour of agents are visualized in Figure ?, Figure ? and Figure ? for SAC models. All of 3 models exhibit similar walking characteristics.

SAC model performs better than TD3 in general, as seen in Figure ? and Figure ?.

In addition, as seen from the moving average points, SAC agents are superior (Figure ??, Figure ??).

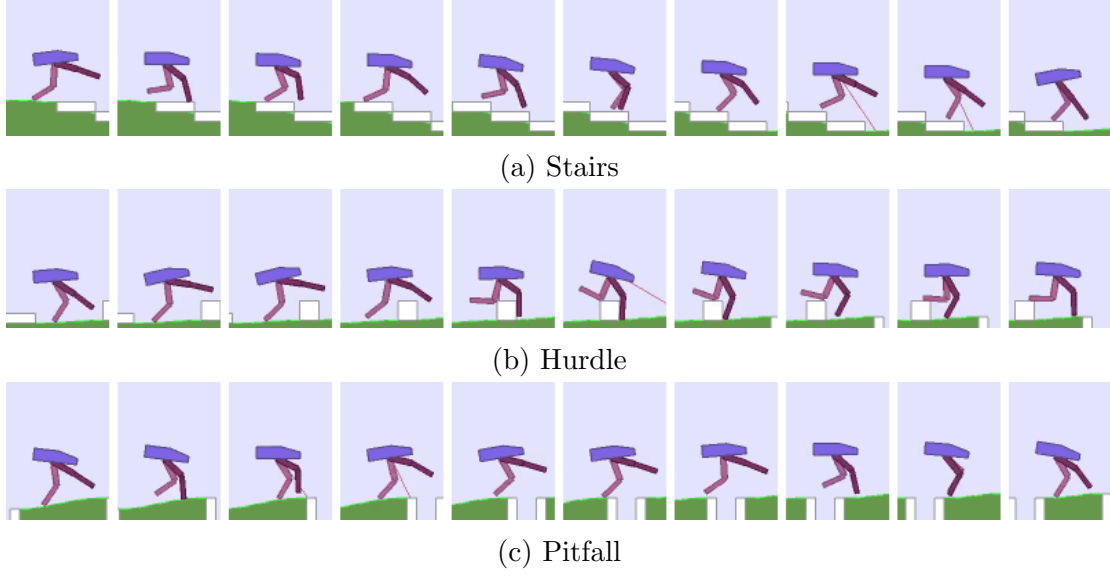


Figure 8: Walking Simulation of RFFNN model at best version with SAC

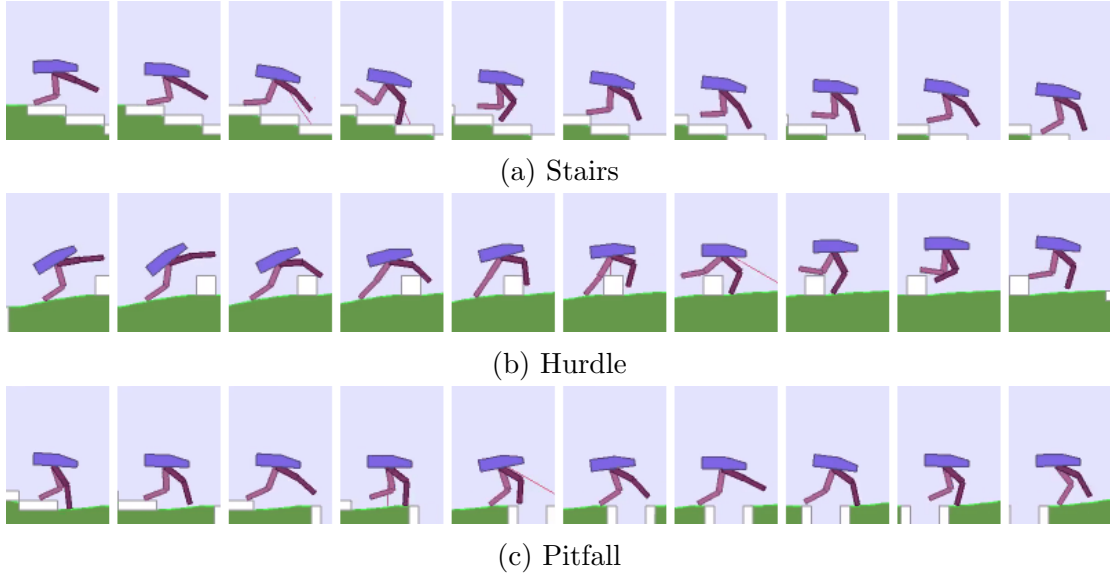


Figure 9: Walking Simulation of LSTM-12 model at best version with SAC

These results are not enough to conclude on a superior neural network for all RL problems, because there are other factors such as DRL algorithm, number of episodes, network size etc. However, networks are designed to have similar sizes and good model requires to converge in less episodes. As a result, LSTM is superior to Transformer for our

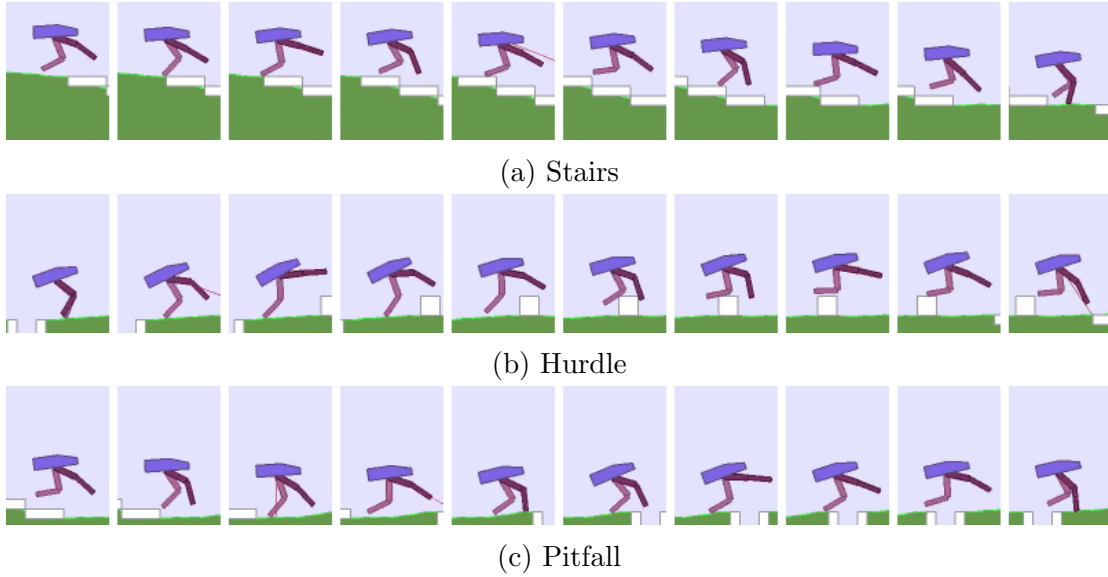


Figure 10: Walking Simulation of Transformer-12 model at best version with SAC

environment. In addition, it is possible to conclude that Transformers can be an option for partially observed RL problems. Note that this is valid where layer normalization is applied before multihead attention and feed-forward layers [?] as opposed to vanilla transformer proposed in [?].

Another result is that incorporating past observations did improve performance significantly since environment is partially observable. Especially increasing history length keeps increasing performance for both LSTM and Transformer models. To address this issue better, we designed Transformer and RFFNN networks to be almost same once multi-head attention layer is removed. We observed significant performance gains as observation history is used as input to controller.

The environment is a difficult one. There are really few available models with solution [?]. Apart from neural networks, there are other factors affecting performance such as RL algorithm, rewarding, exploration etc. In this work, all of them are adjusted such that the environment becomes solvable. Time frequency is reduced for sample efficiency and speed. Also, the agent is not informed for the terminal state when it reaches time limit. Lastly, punishment of falling reduced, so the agent is allowed to learn by mistakes. Those modifications are probably another source of our high performance.

As RL algorithm, TD3 is selected first, since it is suitable for continuous RL. Ornstein-Uhlenbeck noise is used for better exploration since it has momentum, and variance is reduced by time to make agent learn small actions well in later episodes. However, we cannot feed 12 observation history to sequential models since TD3 failed to learn in that case. In addition, SAC is used for learning along with TD3. Results are better compared to those of TD3. SAC policy maximizes randomness (entropy) if agent cannot get sufficient reward and this allows the agent to decide where/when to explore more or less. This way, SAC handles the sparse rewards from the environment better than TD3.

5 Conclusion & Future Work

In RL context, learning in a simulation is an important step for mechanical control. Usually, models are pretrained in simulation environment before learning in reality due to safety and exploration reasons. Today, RL is rarely used in real world applications due to safety and sample inefficiency problems.

In this thesis, bipedal robot walking is investigated by deep reinforcement learning due to complex dynamics in OpenAI Gym’s simulation environment. TD3 and SAC algorithms are used since they are robust and well suited for continuous control. Also, environment is slightly modified by reward shaping, halving simulation frequency, cancelling terminal state information at time limit so that learning becomes easier.

The environment was difficult for exploration. Especially handling big hurdles requires very much exploration. In our results, SAC agents performed better than TD3 since it handles exploration problem by learning how much to explore for a particular state.

As stated in previous chapters, most of the real world environments are partially observable. In *BipedalWalker-Hardcore-v3*, the environment is also partially observable since agent cannot observe behind and it lacks of acceleration sensors, which is better to have for controlling mechanical systems. Therefore, we propose to use Long Short Term Memory (LSTM) and Transformer Neural Networks to capture more information from past observations unlike Residual Feed Forward Neural Network (RFFNN) using a single instant observation as input.

RFFNN model performed well thanks to carefully selected hyperparameters and modifications on the environment. However, sequential models performed much better indicating partial observability is an important issue for our environment. Among sequential models, LSTM performed better compared to Transformer agents. However, note that simple multi-head attention layer added to RFFNN lets Transformer model emerge. Deciding to use observation history with simple attention layer resulted in significant performance gains. This means that input-output design may be more important than neural network design.

Another conclusion is that transformer model performed well enough to say it can be used in DRL problems. It is surprising because it is not successfully used in DRL problems in general except recent architectural developments [?]. In natural language processing, this type of attention models completely replace recurrent models recently, and our results seems promising for this in DRL domain.

Today, all subfields of Deep Learning suffers from lack of analytical methods to design neural networks. It is mostly based on mathematical & logical intuition. Until such methods are developed, it seems that we need to try out several neural networks to get better models, which is the case in our work.

There might be a few possible ways to improve this work. First of all, longer observation history can be used to handle partial observability for LSTM and Transformer models. However, this makes learning slower and difficult and requires more stable RL algorithms and fine tuning on hyperparameters. Secondly, different exploration strategies might be followed. Especially parameter space exploration [?] may perform better since it works

better for environments with sparse reward like our environment. Lastly, advanced neural networks and RL algorithms (specificly designed for environment) may be designed by incorporating domain knowledge.

6 References